SERVERLESS INTELLIGENT FIREWALL

ASLF-OSINT Platform — Complete Development & Deployment Guide

v2.0 — Updated with Verified VM Specifications · Both Proxmox Servers

Md Anisur Rahman Chowdhury · Gannon University · Research Series 1-2-3

Infrastructure	Total VMs	Total RAM	Total Disk
2 Proxmox Servers	6 VMs deployed	24 GB across VMs	380 GiB allocated

Part 0 — Platform Overview & Verified VM Specifications

This guide reflects the actual deployed hardware across both Proxmox servers. All IP addresses, disk sizes, CPU allocations, and RAM values match the provisioned VMs exactly. Every configuration file and command in this document uses these exact specifications.

0.1 Infrastructure Summary

Property	Server 1 — Proxmox (sif1)	Server 2 — Proxmox (sif2)
Proxmox Web UI	`172.16.185.182:8006`	`172.16.184.111:8006`
Cloudflare URL	sif1.marcbd.site	sif2.marcbd.site
Host CPU	i7-4578U · 4 vCPU	i7-4578U · 4 vCPU
Host RAM	15.50 GiB total	15.50 GiB total
Host Disk	95.94 GiB	95.94 GiB
Kernel	6.8.4-2-pve	6.8.4-2-pve
Proxmox Version	pve-manager 8.2.2	pve-manager 8.2.2
Hosted VMs	`VM101 · VM102 · VM103`	`VM201 · VM202 · VM203`
`VM RAM Used`	12 GB total (Server 1 VMs)	12 GB total (Server 2 VMs)
Host RAM Free	~3.5 GB reserved for Proxmox	~3.5 GB reserved for Proxmox
VLAN	None — flat DHCP network	None — flat DHCP network
Subnet	`172.16.185.x / 22 (all VMs)`	`172.16.185.x / 22 (all VMs)`

> **NOTE:** Both Proxmox servers share the same /22 subnet (172.16.184.0/22). All VMs receive IPs via DHCP on vmbr0. No VLANs are used. Services communicate using hostnames defined in /etc/hosts on each VM.

0.2 VM Specifications — Server 1 (172.16.185.182)

VM101 Server 1 `172.16.185.182`	**sif-core** *Super Control System — FastAPI · PostgreSQL · Redis · Celery*

IP: `172.16.185.97/22`
RAM: 4 GB (4096 MiB) **CPU:** 2 vCPU **Disk:** 60 GiB
Services: *FastAPI :8000 · PostgreSQL :5432 · Redis :6379 · Celery workers*

VM102
Server 1
`172.16.185.182`

sif-ai-engine

ASLF-OSINT AI Engine — XGBoost · BiGRU · PPO · FedNova · OSINT Scheduler
IP: `172.16.185.230/22`
RAM: 6 GB (6144 MiB) **CPU:** 2 vCPU **Disk:** 100 GiB
Services: *FastAPI :8001 · MLflow :5000 · ONNX exports · PyTorch (CPU)*

VM103
Server 1
`172.16.185.182`

sif-dashboard

Super Control Dashboard — React / Next.js 14 + Nginx reverse proxy
IP: `172.16.185.234/22`
RAM: 2 GB (2048 MiB) **CPU:** 1 vCPU **Disk:** 50 GiB
Services: *Next.js :3000 · Nginx :80 · Live threat feed · Client management UI*

0.3 VM Specifications — Server 2 (172.16.184.111)

VM201
Server 2
`172.16.184.111`

sif-client-host

Client Sub-System Host — Docker host for all client firewall instances
IP: `172.16.185.231/22`
RAM: 6 GB (6144 MiB) **CPU:** 2 vCPU **Disk:** 60 GiB
Services: *Provisioner API :8500 · Docker Engine · per-client Docker Compose stacks*

VM202
Server 2
`172.16.184.111`

sif-broker

Event Broker — model updates · policy changes · OSINT broadcast
IP: `172.16.185.236/22`
RAM: 2 GB (2048 MiB) **CPU:** 1 vCPU **Disk:** 50 GiB
Services: *RabbitMQ AMQP :5672 · Mgmt UI :15672 · Redis Streams :6379*

VM203
Server 2
172.16.184.111

sif-monitor

Observability Stack — Prometheus · Grafana · Loki · Node Exporter

IP: **172.16.185.167/22**
RAM: 4 GB (4096 MiB) **CPU:** 2 vCPU **Disk:** 60 GiB
Services: *Prometheus :9090 · Grafana :3000 · Loki :3100 · Node Exporter :9100*

0.4 Resource Utilisation Summary

VM ID	Hostname	IP Address	RAM	vCPU	Disk	Server
VM101	sif-core	172.16.185.97	4 GB	2	60 GiB	Server 1
VM102	sif-ai-engine	172.16.185.230	6 GB	2	100 GiB	Server 1
VM103	sif-dashboard	172.16.185.234	2 GB	1	50 GiB	Server 1
VM201	sif-client-host	172.16.185.231	6 GB	2	60 GiB	Server 2
VM202	sif-broker	172.16.185.236	2 GB	1	50 GiB	Server 2
VM203	sif-monitor	172.16.185.167	4 GB	2	60 GiB	Server 2
TOTAL	—	—	24 GB	10	380 GiB	Both Servers

Part 1 — Network Configuration & /etc/hosts Registry

Since all VMs receive IPs via DHCP and no VLANs are configured, you must add this block to /etc/hosts on every VM so services resolve each other by hostname. Run this on all 6 VMs before installing any service.

```
# Run on ALL 6 VMs — paste exactly as shown
sudo tee -a /etc/hosts << 'EOF'

# ============================================
# SIF Platform — VM Registry (verified IPs)
# ============================================
172.16.185.97   sif-core
172.16.185.230  sif-ai-engine
172.16.185.234  sif-dashboard
172.16.185.231  sif-client-host
172.16.185.236  sif-broker
172.16.185.167  sif-monitor
EOF
```

> **NOTE:** If a DHCP lease changes, update /etc/hosts on all VMs and restart the affected service. Consider asking your network admin to configure DHCP reservations for all 6 MAC addresses to make IPs permanent.

1.1 Service-to-Service Connection Strings

Service	Connection String (used in config files)	Used By
PostgreSQL	`postgresql://sif_user:SIF_DB_Pass2024!@sif-core:5432/sif_platform`	VM101, VM102
Redis (core)	`redis://:SIF_Redis2024!@sif-core:6379`	VM102, VM201, VM203
RabbitMQ AMQP	`amqp://sifadmin:SIF_Rabbit2024!@sif-broker:5672`	VM101, VM102, VM201
Redis (broker)	`redis://:SIF_Redis2024!@sif-broker:6379`	VM201, VM202
AI Engine API	`http://sif-ai-engine:8001`	VM101, VM201
Core API	`http://sif-core:8000`	VM103, VM201

Provisioner	`http://sif-client-host:8500`	`VM101`
Prometheus	`http://sif-monitor:9090`	`VM203 (scrapes all)`
Grafana	`http://sif-monitor:3000`	`VM103 (embed)`

Part 2 — Base OS Setup (All 6 VMs)

Run this on every VM before any service-specific script. Pass the VM hostname as the argument.

```
# Example on VM101:  sudo bash 00_base_all_vms.sh sif-core
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y curl wget git vim net-tools htop unzip \
  ca-certificates gnupg lsb-release qemu-guest-agent

sudo timedatectl set-timezone America/New_York
sudo hostnamectl set-hostname <HOSTNAME>    # e.g. sif-core
sudo systemctl enable --now qemu-guest-agent

# Then add /etc/hosts block from Part 1
```

2.1 Execution Order

Order	VM	IP Address	Why This Order
1st	`VM202  sif-broker`	`172.16.185.236`	RabbitMQ must exist before anything connects to it
2nd	`VM101  sif-core`	`172.16.185.97`	PostgreSQL + Redis + API — other VMs call this
3rd	`VM102  sif-ai-engine`	`172.16.185.230`	Slowest install (~10 min, PyTorch CPU download)
4th	`VM201  sif-client-host`	`172.16.185.231`	Docker + Provisioner API
5th	`VM203  sif-monitor`	`172.16.185.167`	Prometheus, Grafana, Loki
6th	`VM103  sif-dashboard`	`172.16.185.234`	Dashboard last — needs all APIs already running

Part 3 — VM101: sif-core (172.16.185.97)

Super Control System backend. Manages all corporate client accounts, orchestrates provisioning on VM201, stores all threat events in PostgreSQL, and exposes the REST API consumed by the dashboard and client sub-systems.

3.1 Hardware Allocation

```
VM ID:     101          Hostname:  sif-core
Server:    172.16.185.182  (Proxmox Server 1)
IP:        172.16.185.97/22
RAM:       4096 MiB  (4 GB)
vCPU:      2 cores
Disk:      60 GiB   (PostgreSQL data + Redis AOF + Celery logs)
OS:        Ubuntu Server 22.04 LTS
Username:  sifadmin  /  Password: MARC@151995$
```

3.2 Installation Commands

```
# PostgreSQL 15
sudo apt-get install -y postgresql postgresql-contrib libpq-dev
sudo systemctl enable --now postgresql

# Create database and user
sudo -u postgres psql << 'SQL'
CREATE USER sif_user WITH ENCRYPTED PASSWORD 'SIF_DB_Pass2024!';
CREATE DATABASE sif_platform OWNER sif_user;
GRANT ALL PRIVILEGES ON DATABASE sif_platform TO sif_user;
\q
SQL

# Redis
sudo apt-get install -y redis-server
sudo sed -i 's/^bind 127.0.0.1/bind 0.0.0.0/' /etc/redis/redis.conf
sudo sed -i 's/^# requirepass.*/requirepass SIF_Redis2024!/' /etc/redis/redis.conf
sudo systemctl enable --now redis-server

# Python 3.11 + FastAPI stack
sudo apt-get install -y python3.11 python3.11-venv python3.11-dev build-essential
sudo python3.11 -m venv /opt/sif-core/venv
sudo /opt/sif-core/venv/bin/pip install fastapi "uvicorn[standard]" \
  sqlalchemy alembic psycopg2-binary redis celery pika aio-pika \
  httpx "python-jose[cryptography]" passlib bcrypt pydantic loguru
```

3.3 systemd Service

```
# /etc/systemd/system/sif-core.service
[Unit]
Description=SIF Super Control System Backend
After=network.target postgresql.service redis.service
```

```
[Service]
User=sifadmin
WorkingDirectory=/opt/sif-core
Environment=PYTHONPATH=/opt/sif-core
ExecStart=/opt/sif-core/venv/bin/uvicorn app.main:app \
          --host 0.0.0.0 --port 8000 --workers 2
Restart=always

[Install]
WantedBy=multi-user.target
```

3.4 Service Endpoints

- REST API: http://172.16.185.97:8000
- Swagger UI: http://172.16.185.97:8000/docs
- Health: http://172.16.185.97:8000/health
- PostgreSQL: sif-core:5432 / db: sif_platform / user: sif_user
- Redis: sif-core:6379 / password: SIF_Redis2024!

Part 4 — VM102: sif-ai-engine (172.16.185.230)

The autonomous learning brain. Implements all five paradigms from Research 3: XGBoost + BiGRU base detection (98.7% accuracy), PPO reinforcement learning, DAWMA continual drift detection (12.4 min adaptation), Prototypical Networks meta-learning (94.3% zero-day), and FedNova federated learning (99.8% policy consistency). Also ingests OSINT from AlienVault OTX, VirusTotal, and MISP every 60 minutes.

4.1 Hardware Allocation

```
VM ID:     102          Hostname:  sif-ai-engine
Server:    172.16.185.182  (Proxmox Server 1)
IP:        172.16.185.230/22
RAM:       6144 MiB  (6 GB)   -- largest allocation; handles ML training in RAM
vCPU:      2 cores
Disk:      100 GiB  -- largest disk; model artifacts + MLflow + OSINT cache
OS:        Ubuntu Server 22.04 LTS
Username:  sifadmin  /  Password: MARC@151995$
```

> **NOTE:** VM102 has 100 GiB disk — the largest of all VMs — because: trained XGBoost/BiGRU artifacts consume 2–5 GB per run; MLflow experiment tracking grows continuously; CICIDS2017 dataset is 1.7 GB; ONNX model exports are distributed to all client sub-systems on VM201.

4.2 ML Stack Installation

```
sudo apt-get install -y python3.11 python3.11-venv python3.11-dev build-essential
sudo python3.11 -m venv /opt/sif-ai/venv

# Install in batches — 6 GB RAM is sufficient for CPU-only PyTorch
sudo /opt/sif-ai/venv/bin/pip install numpy pandas scikit-learn xgboost joblib

sudo /opt/sif-ai/venv/bin/pip install torch \
  --index-url https://download.pytorch.org/whl/cpu

sudo /opt/sif-ai/venv/bin/pip install "stable-baselines3[extra]" gymnasium

sudo /opt/sif-ai/venv/bin/pip install fastapi "uvicorn[standard]" httpx \
  redis aio-pika pika mlflow shap loguru schedule onnx onnxruntime
```

4.3 OSINT API Keys

```
# Add to /etc/environment on VM102
OTX_API_KEY=your_alienvault_otx_key
VT_API_KEY=your_virustotal_key
MISP_URL=https://your-misp-instance.org
MISP_KEY=your_misp_key

source /etc/environment
```

4.4 Service Endpoints

- AI Detection API: http://172.16.185.230:8001
- Swagger UI: http://172.16.185.230:8001/docs
- MLflow Tracking: http://172.16.185.230:5000
- Model exports: /opt/sif-ai/exports/ (ONNX → VM201 clients via RabbitMQ)

Part 5 — VM202: sif-broker (172.16.185.236)

Event broker — the communication backbone. Must be the first service started. RabbitMQ fanout exchanges broadcast model updates and policy changes to every client sub-system simultaneously. Redis Streams handles fast in-memory event queuing.

5.1 Hardware Allocation

```
VM ID:      202          Hostname:  sif-broker
Server:     172.16.184.111  (Proxmox Server 2)
IP:         172.16.185.236/22
RAM:        2048 MiB  (2 GB)   -- sufficient; RabbitMQ is lightweight
vCPU:       1 core
Disk:       50 GiB
OS:         Ubuntu Server 22.04 LTS
Username:   sifadmin  /  Password: MARC@151995$
```

5.2 RabbitMQ Exchanges

Exchange	Type	Publisher	Subscribers
model_updates	fanout	`VM102 AI Engine`	All client containers on VM201
policy_updates	fanout	`VM101 Core API`	All client containers on VM201
threat_intel	topic	`VM102 AI Engine`	`VM101 Core API (ingestion)`
client_events	direct	Client containers	`VM101 Core API`

5.3 Service Endpoints

- RabbitMQ AMQP: amqp://172.16.185.236:5672
- RabbitMQ Mgmt: http://172.16.185.236:15672 Login: sifadmin / SIF_Rabbit2024!
- Redis Streams: redis://172.16.185.236:6379 Password: SIF_Redis2024!

Part 6 — VM201: sif-client-host (172.16.185.231)

Docker host for all client firewall sub-systems. When a corporate client is provisioned via the Super Control Dashboard, VM101 calls VM201's Provisioner API, which spins up a dedicated 3-container Docker Compose stack for that client. Each stack is fully isolated from other clients.

6.1 Hardware Allocation

```
VM ID:     201          Hostname:  sif-client-host
Server:    172.16.184.111  (Proxmox Server 2)
IP:        172.16.185.231/22
RAM:       6144 MiB  (6 GB)   -- shared across all client Docker stacks
vCPU:      2 cores
Disk:      60 GiB   -- grows with clients (~500 MB per client stack)
OS:        Ubuntu Server 22.04 LTS
Username:  sifadmin  /  Password: MARC@151995$
```

> **NOTE:** With 6 GB RAM, VM201 can host approximately 8–12 concurrent client Docker stacks. Each stack consumes ~300–500 MB RAM (3 containers: firewall + dashboard + agent). Docker handles disk deduplication efficiently.

6.2 Docker Installation

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu jammy stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo usermod -aG docker sifadmin
sudo systemctl enable --now docker
```

6.3 Client Provisioning Flow

1. Admin clicks "New Client" in Super Control Dashboard → POST /api/v1/clients/provision on VM101 (sif-core)
2. VM101 calls: POST http://sif-client-host:8500/provision with {client_id, subdomain, api_key}
3. VM201 renders Docker Compose template, substitutes CLIENT_ID and API_KEY
4. VM201 runs: docker compose -f /opt/sif-client-host/stacks/{CLIENT_ID}/docker-compose.yml up -d
5. Three containers start: sif-{CLIENT_ID}-firewall · sif-{CLIENT_ID}-dashboard · sif-{CLIENT_ID}-agent
6. Client receives dashboard URL and API key instantly from VM101 response

6.4 Service Endpoints

- Provisioner API: http://172.16.185.231:8500
- API Docs: http://172.16.185.231:8500/docs
- Stacks directory: /opt/sif-client-host/stacks/{CLIENT_ID}/

Part 7 — VM203: sif-monitor (172.16.185.167)

Full observability stack. Prometheus scrapes metrics from all 6 VMs every 15 seconds. Grafana provides live dashboards for the global Super Control view and each client sub-dashboard (embedded via iframe). Loki aggregates structured logs from all services.

7.1 Hardware Allocation

```
VM ID:     203          Hostname:  sif-monitor
Server:    172.16.184.111  (Proxmox Server 2)
IP:        172.16.185.167/22
RAM:       4096 MiB  (4 GB)   -- Prometheus TSDB in-memory + Grafana rendering
vCPU:      2 cores
Disk:      60 GiB   -- 30-day Prometheus TSDB retention (~2 GB/day estimated)
OS:        Ubuntu Server 22.04 LTS
Username:  sifadmin  /  Password: MARC@151995$
```

7.2 Prometheus Scrape Configuration

```
# /etc/prometheus/prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "sif-core"
    static_configs: [{targets: ["sif-core:8000"]}]

  - job_name: "sif-ai-engine"
    static_configs: [{targets: ["sif-ai-engine:8001"]}]

  - job_name: "sif-client-host"
    static_configs: [{targets: ["sif-client-host:8500"]}]

  - job_name: "node-exporter"
    static_configs:
      - targets:
          - "sif-core:9100"          # VM101  172.16.185.97
          - "sif-ai-engine:9100"     # VM102  172.16.185.230
          - "sif-dashboard:9100"     # VM103  172.16.185.234
          - "sif-client-host:9100"   # VM201  172.16.185.231
          - "sif-broker:9100"        # VM202  172.16.185.236
          - "sif-monitor:9100"       # VM203  172.16.185.167
```

7.3 Service Endpoints

- Prometheus: http://172.16.185.167:9090
- Grafana: http://172.16.185.167:3000 Login: sifadmin / SIF_Grafana2024!
- Loki: http://172.16.185.167:3100
- Node Exporter: :9100 on every VM

Part 8 — VM103: sif-dashboard (172.16.185.234)

The Super Control Dashboard — a Next.js 14 application served by Nginx. Provides the global administrative view: all client statuses, aggregated threat intelligence, real-time detection feed, model version across clients, and OSINT feed health.

8.1 Hardware Allocation

```
VM ID:     103          Hostname:  sif-dashboard
Server:    172.16.185.182  (Proxmox Server 1)
IP:        172.16.185.234/22
RAM:       2048 MiB  (2 GB)   -- Next.js SSR + Nginx; lightest VM
vCPU:      1 core
Disk:      50 GiB   -- no heavy storage; just app code and Nginx logs
OS:        Ubuntu Server 22.04 LTS
Username:  sifadmin  /  Password: MARC@151995$
```

8.2 Installation

```
# Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs nginx

# Build Next.js dashboard
cd /opt/sif-dashboard
npm install && npm run build

# Start services
sudo systemctl enable --now sif-dashboard nginx
```

8.3 Nginx Reverse Proxy Config

```
# /etc/nginx/sites-available/sif-dashboard
server {
    listen 80 default_server;
    server_name _;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }

    location /api/sif/ { proxy_pass http://sif-core:8000/api/v1/; }
    location /api/ai/  { proxy_pass http://sif-ai-engine:8001/; }
}
```

8.4 Service Endpoints

- Dashboard (Nginx): http://172.16.185.234:80
- Next.js direct: http://172.16.185.234:3000
- Via Cloudflare: https://dashboard.sif.marcbd.site (through sif1.marcbd.site tunnel)

Part 9 — Platform Startup Sequence & Validation

Always start services in this exact order. Dependencies flow: broker → core → AI → clients → monitoring → dashboard.

Step 1 — VM202: sif-broker (172.16.185.236) — START FIRST

```
ssh sifadmin@172.16.185.236
sudo systemctl start rabbitmq-server redis-server
# Verify:
sudo systemctl is-active rabbitmq-server redis-server
# Expected: active  active
```

Step 2 — VM101: sif-core (172.16.185.97)

```
ssh sifadmin@172.16.185.97
sudo systemctl start postgresql redis-server sif-core
# Verify:
curl -s http://localhost:8000/health
# Expected: {"status": "healthy"}
```

Step 3 — VM102: sif-ai-engine (172.16.185.230)

```
ssh sifadmin@172.16.185.230
sudo systemctl start sif-ai-engine sif-mlflow
# Verify:
curl -s http://localhost:8001/health
# Expected: {"status": "healthy", "model_version": "3.0.0-base"}
```

Step 4 — VM201: sif-client-host (172.16.185.231)

```
ssh sifadmin@172.16.185.231
sudo systemctl start docker sif-provisioner
# Verify:
curl -s http://localhost:8500/health
# Expected: {"status": "healthy"}
```

Step 5 — VM203: sif-monitor (172.16.185.167)

```
ssh sifadmin@172.16.185.167
sudo systemctl start prometheus grafana-server loki node-exporter
# Prometheus: http://172.16.185.167:9090
# Grafana:    http://172.16.185.167:3000  (sifadmin / SIF_Grafana2024!)
```

Step 6 — VM103: sif-dashboard (172.16.185.234) — START LAST

```
ssh sifadmin@172.16.185.234
sudo systemctl start sif-dashboard nginx
# Dashboard: http://172.16.185.234
```

9.1 End-to-End Smoke Test

```
# From any VM — provision a test client
curl -X POST "http://sif-core:8000/api/v1/clients/provision?
name=TestCorp&email=test@testcorp.com"

# Test AI detection with a synthetic DoS flow
curl -X POST http://sif-ai-engine:8001/detect \
  -H "Content-Type: application/json" \
  -d '{"features":[0,10000,0,0,1500000,9800,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
         0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
         0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0], "source_ip": "192.168.1.100"}'

# Expected: {"attack_type": "DoS", "confidence": 0.97,
#            "trust_score": 0.3, "action_taken": "block_ip"}
```

Appendix A — Complete Service Endpoint Reference

VM	Hostname	IP Address	Port	Service
VM101	sif-core	172.16.185.97	:8000	FastAPI REST API (Swagger: /docs)
VM101	sif-core	172.16.185.97	:5432	PostgreSQL (db: sif_platform)
VM101	sif-core	172.16.185.97	:6379	Redis (pw: SIF_Redis2024!)
VM102	sif-ai-engine	172.16.185.230	:8001	AI Detection API (Swagger: /docs)
VM102	sif-ai-engine	172.16.185.230	:5000	MLflow Experiment Tracking
VM103	sif-dashboard	172.16.185.234	:80	Super Control Dashboard (Nginx)
VM103	sif-dashboard	172.16.185.234	:3000	Next.js Direct
VM201	sif-client-host	172.16.185.231	:8500	Client Provisioner API (Swagger: /docs)
VM202	sif-broker	172.16.185.236	:5672	RabbitMQ AMQP
VM202	sif-broker	172.16.185.236	:15672	RabbitMQ Mgmt (sifadmin / SIF_Rabbit2024!)
VM202	sif-broker	172.16.185.236	:6379	Redis Streams (pw: SIF_Redis2024!)
VM203	sif-monitor	172.16.185.167	:9090	Prometheus
VM203	sif-monitor	172.16.185.167	:3000	Grafana (sifadmin / SIF_Grafana2024!)
VM203	sif-monitor	172.16.185.167	:3100	Loki Log Aggregation
All VMs	*	*	:9100	Node Exporter (Prometheus metrics)

Appendix B — Credentials Reference

Service	Username	Password	VM / Scope
All VMs SSH	sifadmin	MARC@151995$	All 6 VMs
PostgreSQL	sif_user	SIF_DB_Pass2024!	`VM101`
Redis (sif-core)	(none)	SIF_Redis2024!	`VM101 :6379`
Redis (sif-broker)	(none)	SIF_Redis2024!	`VM202 :6379`
RabbitMQ	sifadmin	SIF_Rabbit2024!	`VM202 :15672`
Grafana	sifadmin	SIF_Grafana2024!	`VM203 :3000`

> **NOTE:** Rotate all passwords before exposing any service outside the 172.16.x.x subnet via Cloudflare tunnel. Store in a password manager — never commit to Git.

Appendix C — Research Performance Targets

Key metrics from Research 3 (ASLF-OSINT) that this deployment is engineered to achieve:

Metric	Target	Source / Method
Detection accuracy	98.7%	CICIDS2017 + CIC-IDS2018 + UNSW-NB15 + NSL-KDD
Average inference latency	87 ms	AWS Lambda · Azure Functions · GCP Functions
Zero-day detection (5-shot)	94.3%	Prototypical Networks meta-learning (5 samples)
Drift adaptation time	12.4 minutes	DAWMA dual-window + SSF continual learning
Federated policy consistency	99.8%	FedNova across AWS + Azure + GCP nodes
Analyst trust score (SHAP)	91.2%	SHAP + LIME explainability integration
Cost per 10,000 invocations	$0.25 USD	AWS Lambda serverless pricing baseline